

CO1_AT3 – CODING CHALLENGE

Question 1: Simulate CPU Fetch, Decode, Execute, Store

Write a program that simulates a CPU performing the operations Fetch, Decode, Execute and Store, for the instruction "ADD 15 20".

Algorithm

- Start.
- FETCH: Read the instruction string "ADD 15 20" into memory.
- DECODE: Parse the string to separate the opcode (ADD) and the two operands (15 and 20).
- EXECUTE: Based on the opcode, perform the arithmetic operation ($15 + 20$).
- STORE: Save the computed result into a register/output.
- Print the result of each stage.
- Stop.

Code (compiler screenshot)

```
q1_cpu_sim.c

$ cat q1_cpu_sim.c
#include <stdio.h>
#include <string.h>

int main() {
    char instruction[50] = "ADD 15 20";
    char opcode[10];
    int op1, op2, result;

    /* ----- FETCH ----- */
    printf("Fetched Instruction\n");

    /* ----- DECODE ----- */
    sscanf(instruction, "%s %d %d", opcode, &op1, &op2);
    printf("Decoded as %s\n", opcode);

    /* ----- EXECUTE ----- */
    if (strcmp(opcode, "ADD") == 0) {
        result = op1 + op2;
    } else if (strcmp(opcode, "SUB") == 0) {
        result = op1 - op2;
    } else {
        result = 0;
    }
    printf("Executed Result = %d\n", result);

    /* ----- STORE ----- */
    int Register = result;
    printf("Stored in Register\n");

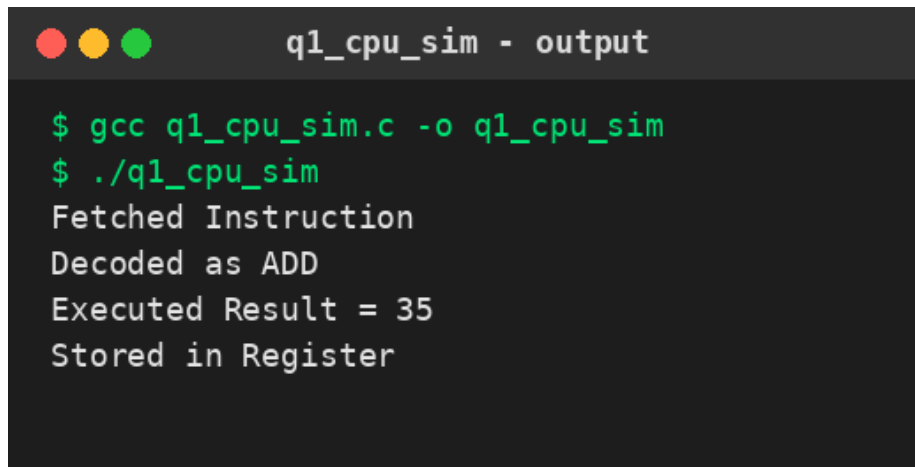
    return 0;
}
```

Sample Input (screenshot)

```
q1 - sample input

Sample Input
-----
ADD 15 20
```

Output (screenshot)



```
q1_cpu_sim - output

$ gcc q1_cpu_sim.c -o q1_cpu_sim
$ ./q1_cpu_sim
Fetched Instruction
Decoded as ADD
Executed Result = 35
Stored in Register
```

Question 2: Count Memory Accesses for a Sequence of Instructions

Given a sequence of CPU instructions, count how many times memory is read from (LOAD) and written to (STORE).

Algorithm

- Start.
- Store the given instruction sequence in an array.
- Initialize counters reads = 0 and writes = 0.
- For each instruction in the sequence:
 - If the instruction is LOAD, increment reads (memory is read).
 - If the instruction is STORE, increment writes (memory is written).
 - Otherwise (ADD, SUB, etc.) do nothing, since these use registers only.
- Print the final values of reads and writes as Memory Reads and Memory Writes.
- Stop.

Code (compiler screenshot)

```
q2_memory_count.c

$ cat q2_memory_count.c
#include <stdio.h>
#include <string.h>

int main() {
    char instructions[10][20] = {"LOAD", "ADD", "STORE", "SUB", "LOAD", "STORE"};
    int n = 6;
    int reads = 0, writes = 0;
    int i;

    for (i = 0; i < n; i++) {
        if (strcmp(instructions[i], "LOAD") == 0) {
            reads++;
            /* LOAD reads from memory */
        } else if (strcmp(instructions[i], "STORE") == 0) {
            writes++;
            /* STORE writes to memory */
        }
        /* ADD / SUB operate on registers only - no memory access */
    }

    printf("Memory Reads = %d\n", reads);
    printf("Memory Writes = %d\n", writes);

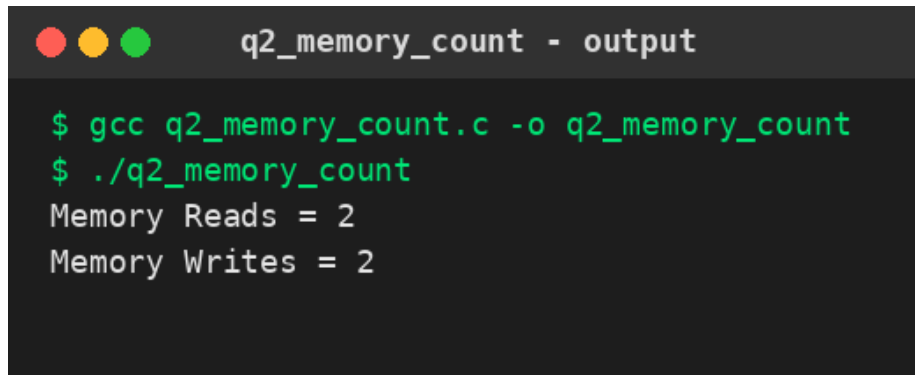
    return 0;
}
```

Sample Input (screenshot)

```
q2 - sample input

Sample Input
-----
LOAD
ADD
STORE
SUB
LOAD
STORE
```

Output (screenshot)

A terminal window with a dark background and light green text. The title bar at the top reads 'q2_memory_count - output'. The terminal shows two commands being executed: '\$ gcc q2_memory_count.c -o q2_memory_count' and '\$./q2_memory_count'. The output of the second command is 'Memory Reads = 2' followed by 'Memory Writes = 2' on the next line.

```
q2_memory_count - output

$ gcc q2_memory_count.c -o q2_memory_count
$ ./q2_memory_count
Memory Reads = 2
Memory Writes = 2
```

Question 3: Simulate the Five-Stage Execution Cycle of a CPU

Write a program that simulates the five stages -- Fetch, Decode, Execute, Memory Access, Write Back -- for a small sequence of instructions (LOAD R1, ADD R2, STORE R3).

Algorithm

- Start.
- Store the instruction sequence (LOAD R1, ADD R2, STORE R3) in an array.
- For each instruction in the array:
 - Print the instruction name.
 - Print the five stages in order: Fetch, Decode, Execute, Memory Access, Write Back.
- Repeat until all instructions are processed.
- Stop.

Code (compiler screenshot)

```
q3_five_stage.c

$ cat q3_five_stage.c
#include <stdio.h>

void runStages(const char *instr) {
    printf("Instruction: %s\n", instr);
    printf("Fetch\n");
    printf("Decode\n");
    printf("Execute\n");
    printf("Memory Access\n");
    printf("Write Back\n\n");
}

int main() {
    const char *program[3] = {"LOAD R1", "ADD R2", "STORE R3"};
    int i;

    for (i = 0; i < 3; i++) {
        runStages(program[i]);
    }

    return 0;
}
```

Sample Input (screenshot)

```
q3 - sample input

Sample Input
-----
LOAD R1
ADD R2
STORE R3
```

Output (screenshot)

```
q3_five_stage - output

$ gcc q3_five_stage.c -o q3_five_stage
$ ./q3_five_stage
Instruction: LOAD R1
Fetch
Decode
Execute
Memory Access
Write Back

Instruction: ADD R2
Fetch
Decode
Execute
Memory Access
Write Back

Instruction: STORE R3
Fetch
Decode
Execute
Memory Access
Write Back
```

Question 4: Exchange the Contents of Registers B and C

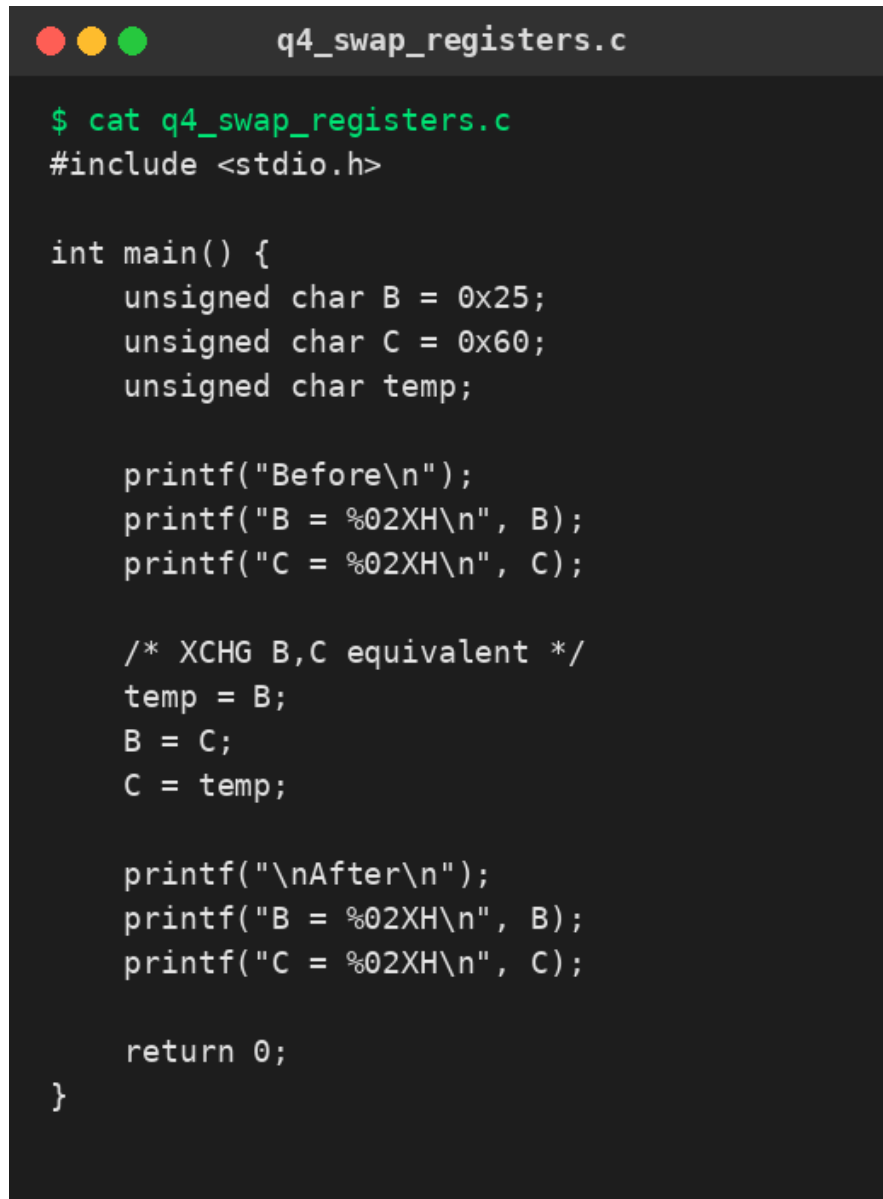
Simulate exchanging the contents of two registers B and C, where initially B = 25H and C = 60H.

Algorithm

- Start.
- Initialize register B = 25H and register C = 60H.
- Print the values of B and C before the exchange.
- Copy the value of B into a temporary variable temp.
- Copy the value of C into B.
- Copy the value of temp into C.
- Print the values of B and C after the exchange.

- Stop.

Code (compiler screenshot)



```
q4_swap_registers.c

$ cat q4_swap_registers.c
#include <stdio.h>

int main() {
    unsigned char B = 0x25;
    unsigned char C = 0x60;
    unsigned char temp;

    printf("Before\n");
    printf("B = %02XH\n", B);
    printf("C = %02XH\n", C);

    /* XCHG B,C equivalent */
    temp = B;
    B = C;
    C = temp;

    printf("\nAfter\n");
    printf("B = %02XH\n", B);
    printf("C = %02XH\n", C);

    return 0;
}
```

Sample Input (screenshot)


```
q4 - sample input

Sample Input
-----
Before Exchange:
B = 25H
C = 60H
```

Output (screenshot)

```
q4_swap_registers - output

$ gcc q4_swap_registers.c -o q4_swap_registers
$ ./q4_swap_registers
Before
B = 25H
C = 60H

After
B = 60H
C = 25H
```

Question 5: Add and Subtract Two 8-bit Numbers in Memory

- (i) Add two 8-bit numbers stored at memory locations 2500H and 2501H and store the result in 2502H.
(ii) Subtract the number at 2501H from the number at 2500H and store the result in 2502H. Sample values used: [2500H] = 14H, [2501H] = 21H.

Algorithm

- Start.
- Declare a memory array large enough to be addressed up to location 2502H.
- Load sample 8-bit values into memory[2500H] and memory[2501H].
- Part (i) Addition:
 - Read the values from memory[2500H] and memory[2501H].
 - Add the two values together.
 - Store the sum into memory[2502H] and print it.
- Part (ii) Subtraction:
 - Read the values from memory[2500H] and memory[2501H].

- Subtract memory[2501H] from memory[2500H].
- Store the result into memory[2502H] and print it.
- Stop.

Code (compiler screenshot)

```
q5_mem_add_sub.c

$ cat q5_mem_add_sub.c
#include <stdio.h>

int main() {
    unsigned char memory[65536] = {0};

    /* Sample 8-bit values stored at 2500H and 2501H */
    memory[0x2500] = 0x14; /* 20 decimal */
    memory[0x2501] = 0x21; /* 33 decimal */

    printf("Part (i): Addition\n");
    printf("[2500H] = %02XH\n", memory[0x2500]);
    printf("[2501H] = %02XH\n", memory[0x2501]);
    memory[0x2502] = memory[0x2500] + memory[0x2501];
    printf("[2502H] = %02XH (Sum)\n\n", memory[0x2502]);

    printf("Part (ii): Subtraction\n");
    printf("[2500H] = %02XH\n", memory[0x2500]);
    printf("[2501H] = %02XH\n", memory[0x2501]);
    memory[0x2502] = memory[0x2500] - memory[0x2501];
    printf("[2502H] = %02XH (2500H - 2501H)\n", memory[0x2502]);

    return 0;
}
```

Sample Input (screenshot)

```
q5 - sample input

Sample Input
-----
Memory[2500H] = 14H
Memory[2501H] = 21H
```

Output (screenshot)

```
q5_mem_add_sub - output

$ gcc q5_mem_add_sub.c -o q5_mem_add_sub
$ ./q5_mem_add_sub
Part (i): Addition
[2500H] = 14H
[2501H] = 21H
[2502H] = 35H (Sum)

Part (ii): Subtraction
[2500H] = 14H
[2501H] = 21H
[2502H] = F3H (2500H - 2501H)
```

Question 6: 8086 Program: Subtraction of Two 8-bit Numbers (Subtract NUM2 from NUM1)

Simulate an 8086-style program that subtracts NUM2 from NUM1. Sample values used: NUM1 = 50H, NUM2 = 20H.

Algorithm

- Start.
- Initialize NUM1 = 50H and NUM2 = 20H (equivalent to loading values into the accumulator and a register in 8086 assembly).
- Print the values of NUM1 and NUM2.
- Perform RESULT = NUM1 - NUM2 (equivalent to the SUB instruction).
- Print the value of RESULT.
- Stop.

Code (compiler screenshot)

```
q6_8086_subtract.c

$ cat q6_8086_subtract.c
#include <stdio.h>

int main() {
    unsigned char NUM1 = 0x50;    /* sample value */
    unsigned char NUM2 = 0x20;    /* sample value */
    unsigned char RESULT;

    printf("NUM1 = %02XH\n", NUM1);
    printf("NUM2 = %02XH\n", NUM2);

    RESULT = NUM1 - NUM2;    /* SUB NUM1, NUM2 */

    printf("RESULT (NUM1 - NUM2) = %02XH\n", RESULT);

    return 0;
}
```

Sample Input (screenshot)

```
q6 - sample input

Sample Input
-----
NUM1 = 50H
NUM2 = 20H
```

Output (screenshot)

```
q6_8086_subtract - output

$ gcc q6_8086_subtract.c -o q6_8086_subtract
$ ./q6_8086_subtract
NUM1 = 50H
NUM2 = 20H
RESULT (NUM1 - NUM2) = 30H
```